73	30	20	26	69	84	76	6
73	30	20	26	69	6	76	84
73	30	20	26	69	6	76	84
6	30	20	26	69	73	76	84
6	30	20	26	69	73	76	84
6	26	20	30	69	73	76	84
6	20	26	30	69	73	76	84
6	20	26	30	69	73	76	84

Aproach: For this lab, we need to set up a sorting algorithm that will sort a list of numbers or words in numerical order, so the left is the smallest and the right is the biggest. It will sort by replacing the biggest number with the last index in the list, then repeating until sorted. To do this, it starts by marking the last index in the list; this is the one that will be swapped with the biggest. The next step is to start iterating through the list for each index and decide which is the biggest. For example, it starts as index 0, which is equal to 5, it then assigns 5 to the variable biggest_so_far, so that it can be stored and replaced later. The index and number will have to be stored in order to perform the "swap" at the end. Now that biggest_so_far is equal to 5, it goes to the next index and compares using less than. So if biggest_so_far is less than the new index's number, the new index's number becomes the biggest so far, and then it continues through the list, replacing that variable for the biggest each time. Once it reaches the marked index, it will decide whether the last index's number is bigger or smaller, if bigger, nothing is swapped; if smaller, it is swapped with the biggest_so_far. Now that a number has been swapped, meaning the furthest right is now the biggest, and wherever the biggest was is now that number, that was in the last index spot, the marker indicating which one needs to be swapped is moved to the left one, and it repeats the check for the biggest. Once it reaches and swaps with index 1, it is done, and can stop, as the smallest doesn't need to be compared with what is in the leftmost spot, as it is the smallest. After it is all done, the new sorted list is printed.

Compare: I think that some of my pros are that I make sure to use fleshed-out variable names, and is easy for a person to read. I think that the AI's code does a better job implementing the sorting sytem a little better, as it doesn't rely on a second list as a crutch to help place the sorted items. I think that mine could be improved by expanding it so it is even easier to read, as well as implementing the code in a way that more closely follows the problem's "aproach" step. I think that AI's could improve in detail, to make sure everything is understood. One of the biggest things I could do to help mine is implement more features from the AI's. Like I forgot to add the check for whether or not the final number is equal to or not. I think that, in general, both solutions would work for this algorithm, but I think that AI might have done it a little more like what would be expected.

```
Pseudocode:
Open JSON with list
List_unsorted <- JSON list
List_sorted <- same list, but will be edited

Set marker as last index
While marker > 0

    SET biggest_so_far <- num at index 0
    SET biggest_num_i <- index where biggest number is

    FOR i in List_unsorted to marker
      IF biggest_so_far < new_num
        SET biggest_so_far <- new_num
        SET biggest_num_i <- new_nem_index

    SET in List_sorted biggest_so_far as value at biggest_num_i
    move marker one left

PUT List_sorted
```

```
Improved code:

Open JSON file                              # O(n)
List <- JSON list                           # O(1)

SET marker_i <- last index of List          # A, O(1)
SET marker_value <- value of last in List   # B, O(1)

WHILE marker_i > 0                          # O(n)

    SET biggest_so_far <- List[0]           # C, O(1)
    SET biggest_num_i <- 0                  # D, O(1)

    FOR i <- 1 TO marker                    # O(n) x O(n)

        SET new_num <- List[i]              # E, O(1)

        IF biggest_so_far < new_num         # O(1)

            SET biggest_so_far <- new_num   # F, O(1)
            SET biggest_num_i <- i          # G, O(1)

        END IF

    END FOR

    Swap biggest_so_far with marker         # H, O(1)
    Swap biggest_num_i with marker[i]       # I, O(1)

    SET marker_i <- marker_i - 1            # J, O(1)

END WHILE

PUT List                                    # K, O(n)

O(n^2)
```

```
AI Pseudocode:
START

Input list

Set marker to last index of list

WHILE marker > 0

    Set biggest_so_far to value at index 0
    Set biggest_index to 0

    FOR index from 1 to marker

        IF value at index > biggest_so_far THEN
            Set biggest_so_far to value at index
            Set biggest_index to index
        END IF

    END FOR

    IF biggest_index is not equal to marker THEN

        Swap value at marker
        with value at biggest_index

    END IF

    Move marker one position to the left

END WHILE

Print sorted list

END
```

Step 1 By Hand: 10 minutes
Step 2 Approach: 8 minutes
Step 3 Pseudocode: 24 minutes
Step 4 Copilot: 10 minutes
Step 5 Compare and Contrast: 7 minutes
Step 6 Update: 27 minutes
Step 7 Trace: 68 minutes
Step 8 Efficiency: 5 minutes

array 26, 06, 90, 55

#	biggest_so_far	biggest_num_i	new_num	marker_i	marker_value	output
A	/	/	/	3	55	26, 06, 90, 55
B	/	/	/	3	55	26, 06, 90, 55
C	26	/	/	3	55	26, 06, 90, 55
D	26	0	/	3	55	26, 06, 90, 55
E	26	0	6	3	55	26, 06, 90, 55
E	26	0	90	3	55	26, 06, 90, 55
F	90	0	90	3	55	26, 06, 90, 55
G	90	2	90	3	55	26, 06, 90, 55
E	90	2	55	3	55	26, 06, 90, 55
H	/	2	/	3	90	26, 06, 55, 90
I	/	/	/	3	90	26, 06, 55, 90
J	/	/	/	2	55	26, 06, 55, 90
C	26	/	/	2	55	26, 06, 55, 90
D	26	0	/	2	55	26, 06, 55, 90
E	26	0	6	2	55	26, 06, 55, 90
E	26	0	55	2	55	26, 06, 55, 90
F	55	0	55	2	55	26, 06, 55, 90
G	55	2	55	2	55	26, 06, 55, 90
E	55	2	55	2	55	26, 06, 55, 90
H	/	2	/	2	55	26, 06, 55, 90
I	/	/	/	2	55	26, 06, 55, 90
J	/	/	/	1	6	26, 06, 55, 90
C	26	/	/	1	6	26, 06, 55, 90
D	26	0	/	1	6	26, 06, 55, 90
E	26	0	6	1	6	26, 06, 55, 90
H	/	0	/	1	26	06, 26, 55, 90
I	/	/	/	1	26	06, 26, 55, 90
J	/	/	/	0	6	06, 26, 55, 90
K	/	/	/	/	/	06, 26, 55, 90